Learning-Based Stochastic Optimization Methods: Denoising Diffusion Probabilistic Models and Diffusion Policy for Robotic Decision-Making

Xinlei Zhang
Dept. of Mechanical and Aerospace Eng.
North Carolina State University
Raleigh, USA
xzhan245@ncsu.edu

Amirhossein Asadi
Dept. of Mechanical and Aerospace Eng.
North Carolina State University
Raleigh, USA
aasadi@ncsu.edu

Yucheng Zhu
Dept. Electrical and Computer Eng.
North Carolina State University
Raleigh, USA
yzhu67@ncsu.edu

***Abstract*—This project investigates learning-based stochastic optimization methods for robot decision-making with a focus on denoising diffusion probabilistic models (DDPMs) and the Diffusion Policy framework. Traditional visuomotor control methods—such as trajectory optimization and model predictive control—often depend on accurate dynamics and hand-crafted objectives, limiting their robustness in complex environments. Imitation learning addresses some of these issues but struggles with multimodal action distributions and long-horizon instability. DDPMs offer an alternative view, interpreting sample generation as iterative stochastic optimization guided by learned score functions. Diffusion Policy extends this idea to action-sequence generation conditioned on observations, enabling stable training, expressive multimodal behaviors, and closed-loop execution. In this work, we analyze these models from an optimization perspective, evaluate their performance on the Push-T manipulation task, and demonstrate that diffusion-based methods provide reliable action refinement, temporal consistency, and robust deployment performance compared to classical imitation learning approaches.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Robotic decision-making can be fundamentally viewed as an *optimization* problem: given the robot's current observations, the optimization objective is to determine an action (or sequence of actions) that minimizes a task-dependent cost, so that the robot can finish the desired task. Classical approaches such as trajectory optimization and model predictive control (MPC) optimize explicit cost functions under known dynamics, but specifying accurate models and objectives becomes difficult in visually rich, contact-heavy manipulation tasks. Learning-based methods, particularly imitation learning, sidestep explicit modeling by directly mapping observations to actions from expert data, yet they introduce their own optimization challenges: multimodal action distributions, long-horizon temporal dependencies, and training instability.

Recent advances in denoising diffusion probabilistic models (DDPMs) [1] and score-based generative models [2] reinterpret sampling as iterative stochastic optimization in a learned gradient field. These models achieve state-of-the-art performance in high-dimensional generative tasks, and improved variants such as DDIM [3] further reduce sampling cost. Building on this progress, diffusion models have been applied to sequential decision-making and reinforcement learning [7], [8], suggesting that diffusion processes can serve as powerful, data-driven optimizers over trajectories.

Diffusion Policy [4], [13] adapts this idea to visuomotor control by representing a policy as a conditional diffusion process over future action sequences. Instead of predicting a single action, the policy iteratively denoises a noisy action sequence conditioned on recent observations and executes a receding-horizon prefix, playing a role analogous to MPC but with an implicit objective encoded by demonstrations. Empirically, Diffusion Policy substantially outperforms behavior cloning (BC), Implicit Behavioral Cloning (IBC) [5], and sequence models such as Behavior Transformers (BeT) [6] on challenging manipulation benchmarks [4], [13]. This report studies DDPMs and Diffusion Policy from the perspective of learning-based stochastic optimization for robot decision-making.

A. Literature Review

Robot visuomotor policy learning lies at the intersection of imitation learning, policy representation, and diffusion-based generative modeling.

1) Imitation learning: Behavior cloning formulates imitation learning as supervised regression from observations to actions. While simple and scalable, BC implicitly assumes that the conditional distribution $p(a \mid o)$ is unimodal and is well represented by its mean. In manipulation tasks, however, there are often multiple valid strategies (e.g., approaching an object from different directions), and the mean of incompatible modes can produce physically invalid actions. BC also suffers from compounding errors over long horizons, since small prediction errors accumulate when the policy is rolled out in closed loop.

Several works extend imitation learning to better capture multimodal behavior. IBC [5] models a conditional energy function and infers actions by minimizing this energy, enabling richer, multi-valued mappings. BeT [6] leverages transformers

TABLE I
COMPARISON OF REPRESENTATIVE POLICY LEARNING APPROACHES FROM AN OPTIMIZATION PERSPECTIVE.

Capability	**DDPMs** [1]	**Score-based** [2]	**IBC** [5]	**BeT** [6]	**Diffuser** [7]	**Diffusion Policy** [4], [13]
Handles multimodal actions	✓	✓	✓	✓	✓	✓
Stable, MSE-style training objective	✓	✓	×	✓	✓	✓
High-dimensional sequence output	✓	✓	✓	✓	✓	✓
Direct focus on robot control	×	×	✓	✓	✓	✓
Closed-loop action sequence generation	×	×	×	×	✓	✓

to model long-horizon behavior in a sequence modeling framework, while Diffuser [7] uses diffusion models to fit trajectory distributions for offline reinforcement learning. These methods improve expressiveness but introduce complex optimization landscapes and can be sensitive to negative sampling, architectural choices, and hyperparameters.

2) Implicit and explicit policy representations: Policy representations can be broadly categorized as explicit or implicit. Explicit policies parameterize $p(a \mid o)$ directly, for instance via mixture density networks, Gaussian policies, or categorical distributions over discretized actions. They are straightforward to train with maximum likelihood objectives but struggle with sharply multimodal or highly non-Gaussian action distributions and do not scale gracefully to very high-dimensional action sequences.

Implicit policies, such as energy-based models and some diffusion-based formulations, define distributions up to a normalizing constant and rely on sampling or inner-loop optimization for inference. IBC [5] is a representative example: the policy does not output an action directly but instead defines an energy landscape over actions that must be minimized. This yields more flexible distributions but makes training and inference more computationally intensive and less stable, largely due to intractable normalizing constants and the need for negative sampling.

Diffusion models offer a compelling compromise. DDPMs [1] and score-based models [2] implicitly represent distributions via learned score functions, yet they are trained using simple mean-squared-error losses on noise prediction rather than contrastive or likelihood-ratio objectives. Diffusion Policy [4], [13] transfers this idea to action space: it learns a gradient field over action sequences conditioned on observations, and inference consists of a fixed number of denoising steps that resemble gradient-based optimization in action space.

B. Challenges

Despite recent progress, several challenges remain central in designing learning-based controllers for manipulation. First, *multimodal action distributions* are ubiquitous in human demonstrations; methods that regress to a single mean action are prone to failure, while mixture models and energy-based methods can be difficult to optimize in high dimensions. Second, policies must generate *high-dimensional, temporally coherent action sequences*: one-step policies accumulate errors over long horizons, whereas sequence models introduce deeper and more complex optimization problems. Third, *optimization stability and robustness* are critical in practice; objectives based on contrastive learning or negative sampling often yield spiky losses and oscillatory performance [4], [5], [13], complicating model selection. Finally, real-world deployment requires *real-time, closed-loop execution* that tolerates perception and computation delays; Diffusion Policy's receding-horizon action sequences have been shown to improve robustness to latency and perturbations [4], [13].

C. Main Questions and Solutions

Diffusion policy [4], [13] solves the aforementioned challenges well as a stochastic optimizer for robotic decision-making. Therefore, in this project, we aim to investigate the success of their approach. Specifically, we focus on three questions:

- How do DDPMs define an optimization problem over noisy latent variables, and how does this formulation extend to conditional action sequences in Diffusion Policy?
- How do the training objectives of DDPMs and Diffusion Policy differ from those of behavior cloning and energy-based imitation learning, and what implications does this have for optimization stability?

Our perspective is to view DDPMs sampling as a fixed-budget stochastic optimization procedure and Diffusion Policy as applying this procedure in action space, conditioned on observations. In this view, the score network acts as a learned gradient oracle, and each denoising step corresponds to a gradient-like update toward high-likelihood trajectories under the demonstration distribution. This optimization lens helps explain why diffusion-based policies can combine stable training, robust closed-loop behavior, and strong multimodal modeling in visuomotor control tasks.

II. PROBLEM FORMULATION: LEARNING-FROM-DEMONSTRATIONS AS A STOCHASTIC OPTIMIZATION PROBLEM

Learning visuomotor control from demonstrations can be formalized as recovering the mapping from observations to expert actions. Diffusion-based approaches extend this perspective by predicting entire future action sequences rather than single actions, enabling the model to capture multimodality, temporal coherence, and long-horizon structure. This section establishes the notation and mathematical framework used throughout the remainder of the report and presents a high-level formulation of the learning problem.

A. Notation

Let the robot operate in discrete time steps $t = 1, 2, \ldots$. At each step, the robot receives an observation $o_t \in \mathcal{O}$ describing the scene (for example, images or proprioception) and an expert provides an action $a_t \in \mathcal{A}$. A demonstration trajectory of length T is written as

$$\tau = \big((o_1, a_1), \ldots, (o_T, a_T)\big), \tag{1}$$

and the dataset of demonstrations is denoted by $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^{N}$.

Following the Diffusion Policy formulation, we introduce three problem horizons. The *observation horizon* T_o specifies the recent history of observations available to the agent at time t,

$$c_t = o_{t-T_o+1:t}. \tag{2}$$

The *prediction horizon* T_p defines the length of the future action sequence the policy is expected to predict,

$$u_t = a_{t:t+T_p-1}. \tag{3}$$

Finally, the *execution horizon* $T_a \leq T_p$ determines how many of these predicted actions are executed before re-planning. This notation provides a structured model of the action-sequence prediction problem that is central to the formulation of diffusion-based visuomotor policies.

B. Stochastic Optimization Problem Formulation

In the scheme of learning-from-demonstrations, the optimization goal can be rephrased as: given the dataset of demonstrations $\mathcal{D}$ as defined earlier, we want to obtain a policy π_θ which maps observations to actions, and minimizes the following objective, where ℓ is typically an MSE or negative log-likelihood loss, measuring the distance between its two input arguments. Therefore, the optimization problem for learning-from-demonstrations can be formulated as,

$$\boxed{\min_{\theta} \ \mathbb{E}_{(o,a)\sim\mathcal{D}}\big[\ell(\pi_\theta(o), a)\big],} \tag{4}$$

where π_θ is the policy represented by a neural network and parameterized by θ. Recall that the actions paired with observations in the dataset $\mathcal{D}$ are produced from human expert. Thus, minimizing the loss in (4) makes the robot act like the human expert.

While effective in simple domains, this one-step formulation implicitly assumes that *the conditional action distribution $p(a_t \mid o_t)$ is unimodal and well approximated by its mean*. In manipulation tasks, however, demonstrations frequently reflect multiple valid strategies, and regressing toward the mean of inconsistent actions leads to behaviors that are rarely physically meaningful. Furthermore, because π_θ predicts only one action based on the current observation, prediction errors accumulate over long horizons, often resulting in unstable closed-loop behavior.

To overcome these limitations, [4], [13] adopts a sequence-based formulation in which they leverage the conditional distribution

$$p_\theta(u_t \mid c_t), \tag{5}$$

where the arguments (2) and (3) are sequences which are defined over a future time interval. This distribution captures both multimodal structure and temporal dependencies. At test time, the agent conditions on recent observations, samples a candidate action sequence $\hat{u}_t$, and executes the first T_a actions before re-planning, yielding a closed-loop receding-horizon control scheme. However, computing p_θ directly is computationally intractable; for now, we assume this distribution is available. As discussed later, DDPMs and Diffusion Policy offer practical, tractable alternatives.

Given this sequence-level distribution p_θ, finding an optimal action sequence is equivalent to maximizing its log-likelihood:

$$\boxed{\max_{u_t} \log p_\theta(u_t \mid c_t),} \tag{6}$$

or in other words, it's equivalent to sampling from that distribution. This perspective connects diffusion-based policies, energy-based policies, and classical planning, all of which seek high-likelihood or low-energy trajectories, but differ in how they perform this search.

C. Problem Statement

The goal of this project is to analyze how DDPMs and Diffusion Policy instantiate a learned stochastic optimization process in action space. We aim to understand how diffusion models refine noisy trajectories through iterative denoising, how conditioning on recent observations shapes the resulting action-sequence distribution, and why diffusion-based objectives exhibit stable optimization behavior compared to behavior cloning and energy-based imitation learning. This problem formulation establishes the foundation for the methods explored in the following sections.

III. Background

Denoising Diffusion Probabilistic Models (DDPMs) [1] are a class of generative models that construct complex data distributions by inverting a gradual noising process. The starting point is a *forward diffusion process* that transforms a clean data sample x_0 into a sequence of increasingly noisy variables $(x_1, \ldots, x_T)$. This process is typically defined as a Markov chain of Gaussian transitions

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t \mathbf{I}\big), \tag{7}$$

where $\{\beta_t\}_{t=1}^{T}$ is a variance schedule with small positive values. By composing these transitions, one obtains a closed-form expression

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \tag{8}$$

with $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$. For sufficiently large T, the marginal distribution $q(x_T)$ is close to an isotropic Gaussian, which makes sampling from the forward process straightforward.

The generative power of DDPMs arises from learning an approximation to the *reverse* Markov chain $p_\theta(x_{t-1} \mid x_t)$ that inverts this noising process. Since the exact reverse conditionals are intractable, they are parameterized as Gaussians whose mean depends on a neural network predicting the added noise.

A widely used parameterization expresses the reverse mean in terms of a noise-prediction model $\epsilon_\theta(x_t, t)$, and training is carried out by minimizing

$$\min_\theta \ \mathbb{E}_{x_0,t,\epsilon}\big[\|\epsilon_\theta(x_t,t) - \epsilon\|_2^2\big], \tag{9}$$

where t is sampled from a predefined distribution over timesteps and x_t is obtained from x_0 and ϵ via the forward process. This simple mean-squared-error objective can be interpreted as a form of denoising score matching [2], in which the network learns the score $\nabla_{x_t} \log q_t(x_t)$ of the corrupted data distribution at each noise level.

From a continuous-time perspective, diffusion models can be linked to stochastic differential equations (SDEs) that describe the evolution of x_t under a variance-preserving or variance-exploding process [2]. The reverse-time SDE involves the score of the data distribution and can be numerically integrated using the learned score network, yielding alternative sampling schemes that generalize the discrete DDPMs updates. Denoising diffusion implicit models (DDIM) [3] further reinterpret these dynamics as non-Markovian, partially deterministic processes, allowing one to trade off the number of reverse steps against sample quality. In all cases, the sampling procedure can be viewed as an iterative refinement process that gradually moves a noisy sample toward high-density regions of the data distribution.

Interpreted through the lens of optimization, each reverse diffusion step resembles a stochastic gradient-ascent update in data space, guided by the learned score field. DDPMs sampling thus implements a fixed-budget stochastic optimization procedure that starts from Gaussian noise and converges, after a finite number of iterations, to samples that follow the learned data distribution. This optimization viewpoint is central to our work. In the next section, we build directly on this background and treat diffusion models as learned stochastic optimizers in *action space*, showing how the Diffusion Policy framework adapts the DDPMs machinery from image generation to visuomotor control and trajectory optimization for robotic decision-making.

IV. Methods: Diffusion Policy

Diffusion Policy [4], [13] adapts the DDPM framework to sequential decision-making by treating future action sequences as the data to be generated. Instead of modeling images, Diffusion Policy operates directly in the *action space* and learns a conditional distribution $p_\theta(u_t \mid c_t)$ whose samples represent feasible, expert-like action trajectories. This section establishes the mathematical correspondence between DDPMs and Diffusion Policy, clarifies what is learned by the model, and explains how the resulting framework implicitly solves the optimization problem defined in (6).

A. From DDPMs to Diffusion Policy

In the DDPM framework, the goal is to learn a generative model over clean data samples x_0. In Diffusion Policy, the role of x_0 is played by the expert action sequence u_t, and the generative process is learned *conditionally* on the observation history c_t. As in DDPMs, the model applies a forward diffusion process to corrupt the clean data and trains a neural network to invert this corruption.

Forward process. Analogous to the DDPM corruption process $q(x_t \mid x_{t-1})$, Diffusion Policy defines a sequence of noisy action variables

$$q(u_t^{(k)} \mid u_t^{(k-1)}) = \mathcal{N}\Big(u_t^{(k)}; \sqrt{1-\beta_k}\, u_t^{(k-1)},\ \beta_k \mathbf{I}\Big) \tag{10}$$

for $k = 1, \ldots, K$, where K is the number of diffusion steps. After sufficiently many steps, the marginal distribution $q(u_t^{(K)})$ becomes approximately Gaussian.

Reverse process and conditioning. In DDPMs, the reverse transition is parameterized by the noise-prediction model $\epsilon_\theta(x_t, t)$. Diffusion Policy introduces an analogous noise-prediction model

$$\epsilon_\theta(u_t^{(k)}, k, c_t), \tag{11}$$

which is additionally conditioned on the observation sequence c_t defined in (2). This conditioning injects scene context and allows the generative model to produce trajectories tailored to the current environment.

The corresponding reverse transition is modeled as

$$p_\theta(u_t^{(k-1)} \mid u_t^{(k)}, c_t) = \mathcal{N}\Big(\mu_\theta(u_t^{(k)}, k, c_t),\ \Sigma_k\Big), \tag{12}$$

where μ_θ is computed from the predicted noise ϵ_θ, following the same construction as in DDPMs.

Training objective and relation to likelihood. Ideally, one would like to learn the conditional distribution $p_\theta(u_t \mid c_t)$ by maximizing its log-likelihood over expert demonstrations, i.e.,

$$\boxed{\max_\theta \ \mathbb{E}_{(u_t,c_t)\sim\mathcal{D}}\big[\log p_\theta(u_t \mid c_t)\big],} \tag{13}$$

and then solving the optimization problem in (6) to compute the optimal action. However, this likelihood is computationally intractable to evaluate directly: computing $\log p_\theta(u_t \mid c_t)$ would require marginalizing over all possible diffusion trajectories $u_t^{(1:K)}$, which is infeasible in high dimensions.

To circumvent this difficulty, DDPMs introduce a variational approximation that yields a tractable *evidence lower bound* (ELBO) on the conditional log-likelihood. Under the DDPM parameterization—Gaussian forward noise, fixed reverse variance, and a noise-prediction network—the ELBO simplifies dramatically, and maximizing it is equivalent to minimizing the noise-prediction mean-squared-error objective [4], [9], [13]

$$\min_\theta \ \mathbb{E}_{u_t,k,\epsilon}\Big[\big\|\epsilon_\theta(u_t^{(k)}, k, c_t) - \epsilon\big\|_2^2\Big], \tag{14}$$

where $u_t^{(k)}$ is obtained through the forward noising process. Thus, (14) is not an arbitrary choice: it is a computationally efficient surrogate for maximizing a variational bound on $\log p_\theta(u_t \mid c_t)$.

In other words, the diffusion-policy training loss learns the conditional distribution $p_\theta(u_t \mid c_t)$ *indirectly* through ELBO maximization, rather than by evaluating or differentiating the true likelihood explicitly. This approach avoids the intractability of computing $p_\theta(u_t \mid c_t)$ in closed form while still ensuring

that the learned model represents a high-quality approximation of the expert action distribution.

Thus, Diffusion Policy is mathematically equivalent to a conditional DDPM in which 1) the "data" are expert action sequences u_t, 2) the conditioning variable is the observation history c_t, and 3) the learned score network implicitly defines the conditional distribution $p_\theta(u_t \mid c_t)$ through its reverse diffusion dynamics. Importantly, Diffusion Policy does not represent p_θ in closed form; instead, it learns the score function whose integration implicitly defines this distribution.

B. Diffusion Policy as a Stochastic Optimization Solver

Recall the optimization problem,

$$\max_{u_t} \log p_\theta(u_t \mid c_t), \tag{15}$$

which seeks an action sequence u_t with the highest log-likelihood under the distribution $p_\theta(u_t \mid c_t)$. Note that although diffusion models do not explicitly compute or maximize the log-density, the reverse diffusion process can be interpreted as a fixed-steps stochastic optimization procedure.

Each reverse step satisfies the approximate update [4], [13],

$$u_t^{(k-1)} \approx u_t^{(k)} + \sigma_k^2 \nabla_{u_t^{(k)}} \log p_\theta(u_t^{(k)} \mid c_t) + \text{stochastic noise}, \tag{16}$$

which resembles a *stochastic gradient-ascent* step on the log-likelihood. Thus, the denoising process implicitly moves samples toward higher-probability regions of the conditional distribution $p_\theta(u_t \mid c_t)$.

Remark: *The above perspective also clarifies that running the reverse process is equivalent to sampling from $p_\theta(u_t \mid c_t)$, because diffusion sampling produces samples concentrated in high-density regions of the learned distribution.*

Connection to the optimization problem. Since each denoising step increases the sample's likelihood, the final prediction $u_t^{(0)}$ can be viewed as an approximate solution to the optimization problem defined in (6):

$$u_t^{(0)} \approx \arg\max_{u_t} \log p_\theta(u_t \mid c_t).$$

Remark: *The Diffusion Policy is not solving this optimization explicitly; instead, the generative sampling process implicitly performs this optimization by repeatedly applying likelihood-increasing updates from (16). This stands in contrast to methods that attempt to represent $p_\theta(u_t \mid c_t)$ in closed form, which would allow explicit maximization or direct sampling from the learned density.*

C. Diffusion Policy in the Robotic Decision-Making Task

Receding-horizon execution. At test time, the agent:

1) samples a noisy initial trajectory $u_t^{(K)} \sim \mathcal{N}(0, I)$,
2) applies the reverse diffusion process to obtain $u_t^{(0)}$,
3) executes the first T_a actions before re-planning.

This receding-horizon scheme naturally incorporates real-time feedback and adapts to changing environments, while leveraging the implicit optimization behavior of the diffusion model to generate high-quality action sequences.

V. Results

In this section, we present the results of applying the diffusion policy to the robotic push-T task. Section V-A describes the simulator environment used for both training and testing. The training results are provided in Section V-B, followed by the corresponding testing results in Section V-C.

A. Simulation Environment and Training Setup

In this subsection, we elaborate the simulation environment, dataset, and neural network architecture used for training and evaluating the diffusion policy on the Push-T manipulation task.

Push-T simulator. This simulation environment is given in the tutorial code [10], which adopts the official 2D Push-T environment implemented with `pymunk` [11], where the agent (robot) controls a circular end-effector to push a rigid T-shaped block into a predefined goal region. The environment is initialized in a 512×512 workspace, with state vectors containing the agent position, block position, and block orientation. The action output is a 2D target position specifying the desired end-effector waypoints. A PD controller with gains $(k_p, k_v) = (100, 20)$ drives the end-effector toward the computed waypoints at a simulation rate of 100 Hz, while control actions are applied at 10 Hz. The goal region is instantiated as a static polygon, and task success is measured via the coverage ratio between the T-block geometry and the goal geometry.

Training Dataset and preprocessing. The training dataset is downloaded from [12] which includes a compressed Zarr archive containing synchronized state-action pairs. For the training purpose, we normalize both actions and observations into $[-1, 1]$, and de-normalize the computed actions when we apply those actions to the robot. This normalization is notably critical for training stability.

Neural network architecture and training protocol. We follow the Diffusion Policy tutorial code [10] for implementation, where a *temporal convolutional network (TCN)* [14] is used as the backbone for the noise-prediction model $\varepsilon_\theta(\cdot)$. TCN enables the model to capture temporal correlations across the prediction horizon, and you can find detailed advantage about this choice in the paper [4], [13]. Concrete CNN architecture and embedding network can be found in the paper [4], [13].

In our implementation following [10], the observations contain the robot position, block position, and block orientation. These observations are first embedded and then fed into the noise-prediction model. The model predicts a sequence of future actions of dimension $T_p \times d_a$, representing the planned target positions for the agent over a short receding horizon.

During training, we use a standard DDPMs scheduler with $K = 100$ denoising iterations. Each training batch samples a timestep $k \in 1, \ldots, K$, corrupts the clean action sequence with Gaussian noise, and trains the network to predict the injected noise via the MSE loss. The model parameters are optimized using the Adam optimizer with a fixed learning rate of 1×10^{-4} and batch size 64. For inference, we employ DDIM to reduce

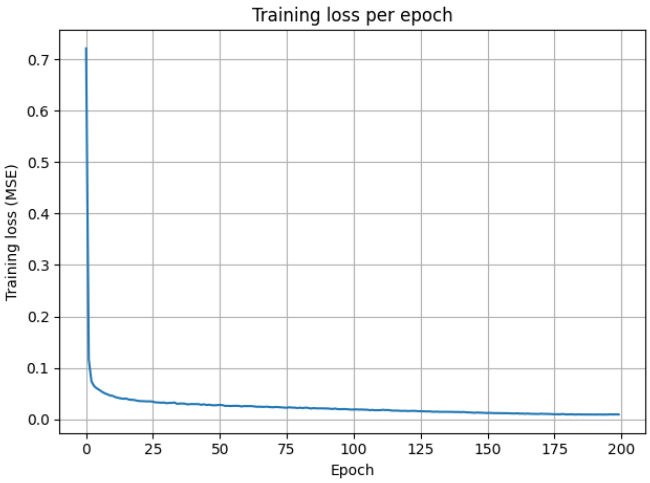

Fig. 1. Training loss (MSE) over 200 epochs, showing rapid early improvement followed by smooth convergence to a low, stable value.

the number of denoising iterations and accelerate online action generation, as did in the tutorial code [10].

B. Training Results

Figure 1 illustrates the evolution of the training loss (measured as mean squared error) over 200 epochs. The loss curve exhibits a sharp decline during the initial epochs, indicating that the model rapidly learns a meaningful mapping between observations and actions early in training. After the initial drop, the loss continues to decrease gradually, eventually stabilizing at a low value. This convergence suggests that the diffusion-based policy successfully learns to predict noise added to action sequences with increasing accuracy, a crucial requirement for denoising diffusion probabilistic models (DDPMs). The smooth nature of the curve further reflects the optimization stability of the DDPM formulation, which is known for avoiding the spiky or oscillatory losses often observed in contrastive or energy-based imitation learning.

Figure 2 presents qualitative results of the robot's performance on the Push-T task at different points in training, culminating in the final epoch (Epoch 199). Each subfigure captures a snapshot of the robot pushing a T-shaped block toward a goal region. The light blue "goal T" represents the desired final position and orientation, while the orange T-block shows the robot's actual progress. The overlaid trajectories trace the block center path, providing insight into how the robot learns to coordinate movements over time. Early in training, the robot's actions appear misaligned and inefficient, with trajectories that overshoot or miss the target. However, as training progresses, the robot increasingly demonstrates coherent and goal-directed behavior. By the final training stage, the diffusion policy reliably generates smooth, collision-free trajectories that push the block accurately into the target zone.

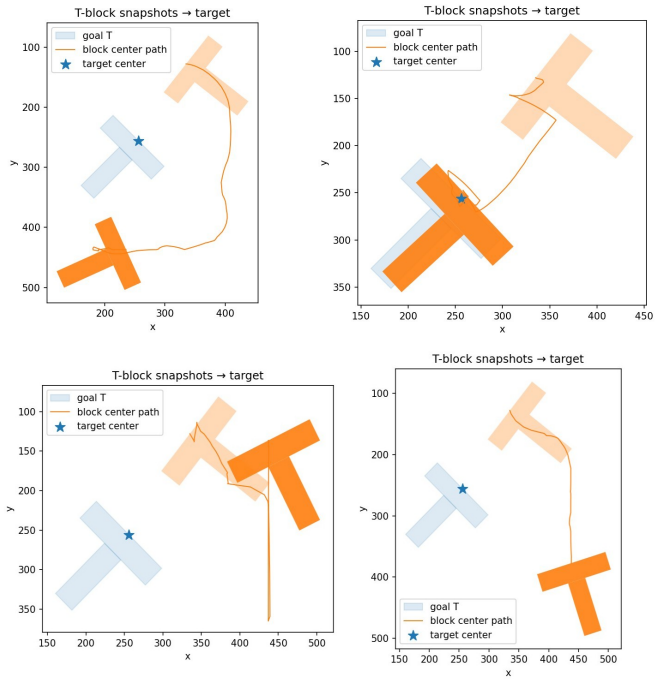

Fig. 2. Snapshots of the robot's performance at different training stages. Early trajectories are inconsistent, while by Epoch 199 the policy reliably pushes the T-block to the target.

C. Push-T Task Testing Results

In this subsection, we evaluate the Diffusion Policy on the Push-T task and analyze: 1) how the policy completes the task, 2) its sensitivity to different initializations, and 3) the variance of its output actions.

Task execution behavior. As shown in Figure 3, the Diffusion Policy successfully drives the robot completing the Push-T task. From the left column of Figure 3, we can observe that the predicted action sequences are smooth and gradually push the T-block toward the goal region. The denoising visualizations on the right column further illustrate the convergence of sampled action sequence at different time step, indicating that the learned denoising model consistently guides the samples toward high-probability pushing behaviors (expert demonstrations).

Sensitivity to initialization. We further evaluate how the robot is driven by the Diffusion Policy under three distinct initial robot–object configurations. As shown in Figure 4, across all cases, the Diffusion Policy successfully generates action sequences that gradually steer the T-block to the goal, showing no failure cases or divergence across the tested initializations. Although the geometric relationships between the robot and the block differ substantially at the start of each trial, the policy consistently adapts by selecting an appropriate pushing direction and forming a stable contact action sequence. These results indicate that the Diffusion Policy is not overly specialized to the training distribution's dominant initializations and exhibits robustness to variations of initializations.

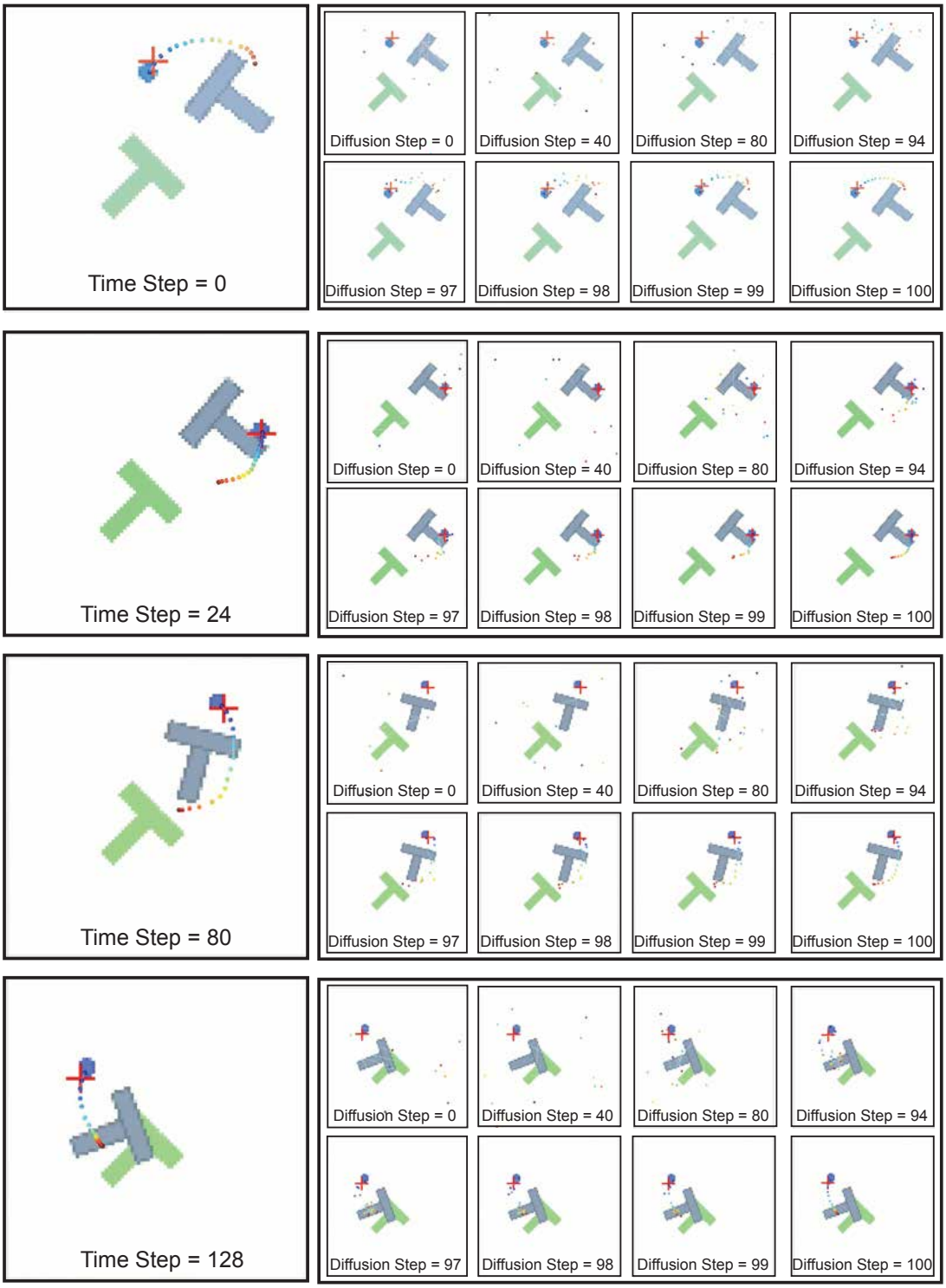

Fig. 3. This figure illustrates the evolution of the robot's manipulation of the T-shaped object over time and the detailed diffusion process at different time steps. **Left column:** Each subfigure corresponds to a different timestep and shows the positions of the robot (blue ball), the T-shaped object (blue gray), its goal (green), and the planned robot trajectory (colored dots). In each subfigure, the color of the dots transitions from blue to red, indicating the planned positions at increasingly future timesteps. Across these four subfigures, the robot gradually pushes the T-shaped object toward its desired goal position and orientation. **Right column:** For each timestep, eight subfigures visualize the diffusion results after successive iterations. For every timestep, the diffusion process begins from random samples in the X–Y space and progressively denoises them, i.e., adjusts the samples toward the optimal solution.

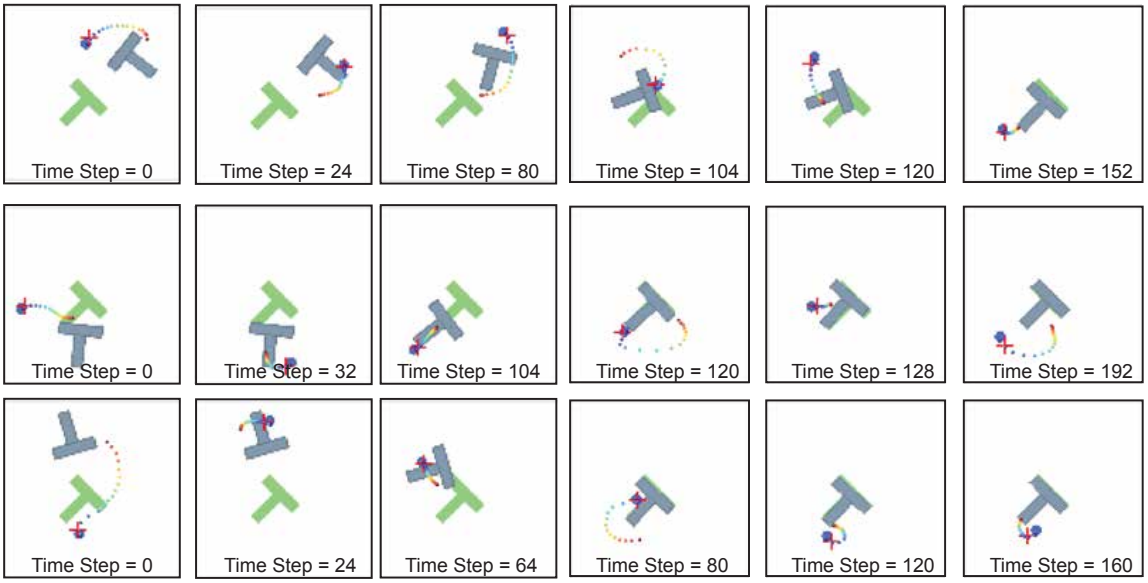

Fig. 4. This figure illustrates the evolution of the robot's manipulation of the T-shaped object over time for three different initializations. Each row corresponds to a distinct initialization, including the positions of the robot (blue ball) and the T-shape object (blue gray) of the object. Within each row, six subfigures (from left to right) depict the positions of the robot and the T-shape object at successive timesteps, showing the robot gradually pushing the T-shaped object toward its goal position and orientation. The consistent success across all three rows indicates that the proposed method is not limited to a particular initialization.

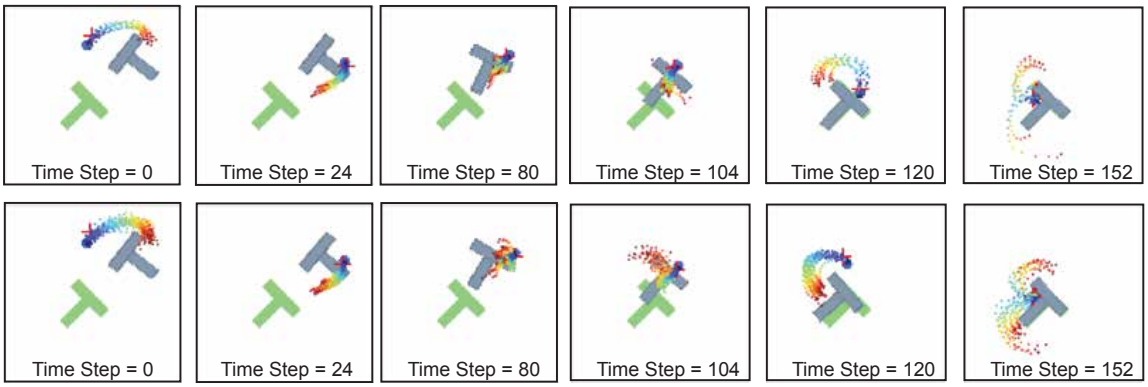

Fig. 5. This figure shows the variance of diffusion-model outputs across time steps under different numbers of repeated diffusion runs `num_diff`. Each row corresponds to an independent trial: the first row shows results with `num_diff= 10`, and the second row shows results with `num_diff= 30`. Although a single diffusion run (`num_diff=1`) is sufficient to generate a planned path for a given trial, we deliberately increase `num_diff` to investigate the variability of the diffusion process. The variance at each time step reflects the inherent randomness in the diffusion process. The last subfigure of each row highlights pronounced multi-modality in the diffusion outputs, where the optimal solution is not unique.

Action-space variance and multimodality. Figure 5 analyzes the variance of the final denoised actions under different numbers of diffusion samples (`num_diff`). Increasing `num_diff` from 10 to 30 illustrates the increased variance of the denoised action distribution. The last column of each row reveals that the policy naturally represents multimodal action distributions—distinct clusters correspond to different valid pushing strategies at the same state. Importantly, each rollout consistently commits to one mode, producing temporally coherent action sequences without oscillatory switching. This confirms that the diffusion-based formulation effectively captures action uncertainty while maintaining stable task execution.

Discussion of limitations. Despite strong performance of the Diffusion Policy, we still observe two limitations. First, the diversity of multimodal predictions depends on the number of diffusion samples and may collapse if insufficient sampling is used. Second, as illustrated in the right column of Figure 3, the denoised action sequences across all four timesteps stabilize only after about 95 diffusion steps. This suggests that the majority of earlier diffusion iterations contribute little refinement, resulting in low computation efficiency.

VI. Conclusion

In this report, we studied denoising diffusion probabilistic models and Diffusion Policy through the lens of learning-based stochastic optimization for robotic decision-making. By

formulating visuomotor control as an optimization problem over future action sequences, we showed that diffusion models can be interpreted as fixed-budget stochastic optimizers whose reverse steps act as gradient-like updates in action space. Within this framework, Diffusion Policy instantiates a conditional diffusion process over trajectories, conditioned on recent observations, and thereby implicitly solves the sequence-level optimization problem without requiring explicit cost functions or tractable likelihoods.

Our experiments on the Push-T manipulation task demonstrate that diffusion-based policies can generate smooth, collision-free trajectories that reliably push the T-shaped block into the goal region. The learned policy exhibits desirable optimization properties: stochastic sampling yields multimodal action proposals, variance across repeated diffusion runs provides a meaningful proxy for model confidence, and temporal convolutional architectures combined with sequence-level diffusion produce temporally consistent action sequences over long horizons. These behaviors align with the theoretical view that reverse diffusion iteratively moves trajectories toward high-likelihood regions of the expert demonstration distribution.

At the same time, our study reveals several limitations and directions for future work. The diversity and robustness of multimodal behaviors depend on the number of diffusion samples and may degrade under tight computational budgets. Moreover, our implementation relies on low-dimensional state observations rather than raw visual inputs, limiting its applicability to more realistic visuomotor settings. Extending Diffusion Policy to image-based observations, integrating more efficient sampling schemes, and comparing against a wider range of optimization-based baselines are promising avenues to further validate and strengthen the interpretation of diffusion models as stochastic optimizers for robot control.

Appendix

Division of Labor

Table II summarizes the division of labor among team members.

TABLE II
TEAM RESPONSIBILITIES

Name	Focus Area
Xinlei	Report writing, Diffusion policy testing
Amirhossein	Report writing, Diffusion policy training
Yucheng	Report writing, Slides making

Submission Files Overview

Please find the detailed code instruction in `Readme.md`.

Submission Folder Structure

```
Submission/
 Readme.md
 Training_Code.py
 diffusion_policy_inference.py
 diffusion_policy_inference_multi.py
 Visualize/
    visualize_denoising.py
    visualize_denoising_multi.py
    visualize_different_multi.py
    visualize_different_seed.py
 data_vis_seed_100k/
 data_vis_seed_200k/
 data_vis_seed_300k/
 data_vis_multi_10/
 data_vis_multi_30/
```

Script Summaries

Core Scripts

- **Training_Code.py**: Trains a Conditional UNet1D diffusion model on the Push-T dataset, saving checkpoints and evaluation metrics.
- **diffusion_policy_inference.py**: Loads a trained model and runs a single inference episode, saving environment images and the full denoising trajectory at each step for visualization.
- **diffusion_policy_inference_multi.py**: Runs inference where the diffusion process is repeated multiple times (e.g., 30) for the same observation at each step, enabling analysis of action consistency and variance.

Visualization Scripts (`Visualize/`)

- **visualize_denoising.py**: Generates a GIF showing the step-by-step denoising process (from noise to final action) for a specific time step in a single run.
- **visualize_denoising_multi.py**: Creates a video visualizing the denoising process for multiple parallel diffusion runs, highlighting convergence across different random seeds.
- **visualize_different_multi.py**: Generates static images overlaying the final predicted action trajectories from all repeats in a multi-run experiment, enabling visualization of the distribution of solutions.
- **visualize_different_seed.py**: Produces static images of the final predicted action trajectory for every time step in a standard single-run experiment.

Data Folders

These folders contain the raw outputs (`images/` and `denoising_actions/`) generated by the inference scripts, required by the visualization tools.

- **data_vis_seed_100k**: Visualization data from a single inference run using random seed 100,000.
- **data_vis_seed_200k**: Visualization data from a single inference run using random seed 200,000.
- **data_vis_seed_300k**: Visualization data from a single inference run using random seed 300,000.
- **data_vis_multi_10**: Visualization data from a multi-run inference experiment with 10 repeats per step.
- **data_vis_multi_30**: Visualization data from a multi-run inference experiment with 30 repeats per step.

REFERENCES

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations (ICLR)*, 2021.

[4] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, 2023.

[5] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Conference on Robot Learning (CoRL)*, 2022.

[6] N. M. Shafiullah, N. Nakhost, B. Eysenbach, N. Heess, S. Gupta, and A. Nair, "Behavior transformers: Cloning k modes with one stone," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] M. Janner, Q. Zhang, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *International Conference on Machine Learning (ICML)*, 2022.

[8] Y. Wang, M. Zhang, and W. Zhang, "Diffusion models for reinforcement learning: A survey," *arXiv preprint arXiv:2311.01223*, 2023.

[9] S. Chan and others, "Tutorial on diffusion models for imaging and vision, " in *Foundations and Trends® in Computer Graphics and Vision*, 2024.

[10] Google Colab, "Push-T Task Diffusion Policy Demo," Accessed: Dec. 2, 2025. [Online]. Available: https://colab.research.google.com/drive/1gxdkgRVfM55zihY9TFLja97cSVZOZq2B?usp=sharing

[11] V. Laxa and contributors, "Pymunk: A Pythonic 2D Physics Library," Accessed: Dec. 2, 2025. [Online]. Available: https://www.pymunk.org/

[12] C. Chi, "Push-T Dataset for Diffusion Policy," 2023. [Online]. Available: https://diffusion-policy.cs.columbia.edu/data/training/

[13] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," in *Robotics: Science and Systems (RSS)*, 2023.

[14] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal Convolutional Networks for Action Segmentation and Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 156–165.